16
Runtime Gameplay Foundation Systems

16.1 Components of the Gameplay Foundation System

Most game engines provide a suite of runtime software components that together provide a framework upon which a game's unique rules, objectives and dynamic world elements can be constructed. There is no standard name for these components within the game industry, but we will refer to them collectively as the engine's *gameplay foundation system*. If a line between engine and game can reasonably be drawn between the game engine and the game itself, then these systems lie just beneath this line. In theory, one can construct gameplay foundation systems that are for the most part game-agnostic. However, in practice, these systems almost always contain genre- or game-specific details. In fact, the line between the engine and the game can probably be best visualized as one big blur—a gradient that arcs across these components as it links the engine to the game. In some game engines, one might even go so far as to consider the gameplay foundation systems as lying entirely *above* the engine-game line. The differences between game engines are most acute when it comes to the design and implementation of their gameplay components. That said, there are a surprising number of common patterns across engines, and those commonalities will be the topic of our discussions here.

Every game engine approaches the problem of gameplay software design a bit differently. However, most engines provide the following major subsystems in some form:

- *Runtime game object model*. This is an implementation of the abstract game object model advertised to the game designers via the world editor.
- *Level management and streaming*. This system loads and unloads the contents of the virtual worlds in which gameplay takes place. In many engines, level data is streamed into memory during gameplay, thus providing the illusion of a large seamless world (when in fact it is broken into discrete chunks).
- *Real-time object model updating*. In order to permit the game objects in the world to behave autonomously, each object must be updated periodically. This is where all of the disparate systems in a game engine truly come together into a cohesive whole.
- *Messaging and event handling*. Most game objects need to communicate with one another. This is usually done via an abstract messaging system. Inter-object messages often signal changes in the state of the game world called *events*. So the messaging system is referred to as the *event system* in many studios.
- *Scripting*. Programming high-level game logic in a language like C or C++ can be cumbersome. To improve productivity, allow rapid iteration, and put more power into the hands of the non-programmers on the team, a scripting language is often integrated into the game engine. This language might be text-based, like Python or Lua, or it might be a graphical language, like Unreal's Blueprints.
- *Objectives and game flow management*. This subsystem manages the player's objectives and the overall flow of the game. This is usually described by a sequence, a tree, or a generalized graph of player objectives. Objectives are often grouped into chapters, especially if the game is highly story-driven as many modern games are. The game flow management system manages the overall flow of the game, tracks the player's accomplishment of objectives and gates the player from one area of the game world to the next as the objectives are accomplished. Some designers refer to this as the "spine" of the game.

Of these major systems, the runtime object model is probably the most complex. It typically provides most, if not all, of the following features:

- *Spawning and destroying game objects dynamically*. The dynamic elements in a game world often need to come and go during gameplay. Health

packs disappear once they have been picked up, explosions appear and then dissipate and enemy reinforcements mysteriously come from around a corner just when you think you've cleared the level. Many game engines provide a system for managing the memory and other resources associated with dynamically spawned game objects. Other engines simply disallow dynamic creation or destruction of game objects altogether.

- *Linkage to low-level engine systems*. Every game object has some kind of linkage to one or more underlying engine systems. Most game objects are visually represented by renderable triangle meshes. Some have particle effects. Many generate sounds. Some animate. Many have collision, and some are dynamically simulated by the physics engine. One of the primary responsibilities of the gameplay foundation system is to ensure that every game object has access to the services of the engine systems upon which it depends.
- *Real-time simulation of object behaviors*. At its core, a game engine is a real-time dynamic computer simulation of an agent-based model. This is just a fancy way of saying that the game engine needs to update the states of all the game objects dynamically over time. The objects may need to be updated in a very particular order, dictated in part by dependencies between the objects, in part by their dependencies on various engine subsystems, and in part because of the interdependencies between those engine subsystems themselves.
- *Ability to define new game object types*. Every game's requirements change and evolve as the game is developed. It's important that the game object model be flexible enough to permit new object types to be added easily and exposed to the world editor. In an ideal world, it should be possible to define a new type of object in an entirely data-driven manner. However, in many engines, the services of a programmer are required in order to add new game object types.
- *Unique object ids*. Typical game worlds contain hundreds or even thousands of individual game objects of various types. At runtime, it's important to be able to identify or search for a particular object. This means each object needs some kind of unique identifier. A human-readable name is the most convenient kind of id, but we must be wary of the performance costs of using strings at runtime. Integer ids are the most efficient choice, but they are very difficult for human game developers to work with. Arguably the best solution is to use hashed string ids (see Section 6.4.3.1) as our object identifiers, as they are as efficient as integers

but can be converted back into string form for ease of reading.

- *Game object queries*. The gameplay foundation system must provide some means of finding objects within the game world. We might want to find a specific object by its unique id, or all the objects of a particular type, or we might want to perform advanced queries based on arbitrary criteria (e.g., find all enemies within a 20 m radius of the player character).
- *Game object references*. Once we've found the objects, we need some mechanism for holding *references* to them, either briefly within a single function or for much longer periods of time. An object reference might be as simple as a pointer to a C++ class instance, or it might be something more sophisticated, like a handle or a reference-counted smart pointer.
- *Finite state machine support*. Many types of game objects are best modeled as finite state machines (FSM). Some game engines provide the ability for a game object to exist in one of many possible states, each with its own attributes and behavioral characteristics.
- *Network replication*. In a networked multiplayer game, multiple game machines are connected together via a LAN or the Internet. The state of a particular game object is usually owned and managed by one machine. However, that object's state must also be *replicated* (communicated) to the other machines involved in the multiplayer game so that all players have a consistent view of the object.
- *Saving and loading / object persistence*. Many game engines allow the current states of the game objects in the world to be saved to disk and later reloaded. This might be done to support a "save anywhere" save-game system or as a way of implementing network replication, or it might simply be the primary means of loading game world chunks that were authored in the world editor tool. Object persistence usually requires certain language features, such as *runtime type identification* (RTTI), *reflection* and *abstract construction*. RTTI and reflection provide software with a means of determining an object's *type*, and what *attributes* and *methods* its class provides, dynamically at runtime. Abstract construction allows instances of a class to be created without having to hard-code the name of the class—a very useful feature when serializing an object instance into memory from disk. If RTTI, reflection and abstract construction are not natively supported in your language of choice, these features can be added manually.

We'll spend the remainder of this chapter delving into each of these subsystems in depth.

16.2 Runtime Object Model Architectures

In the world editor, the game designer is presented with an abstract game object model, which defines the various types of dynamic elements that can exist in the game, how they behave and what kinds of attributes they have. At runtime, the gameplay foundation system must provide a concrete implementation of this object model. This is by far the largest component of any gameplay foundation system.

The runtime object model implementation may or may not bear any resemblance to the abstract tool-side object model. For example, it might not be implemented in an object-oriented programming language at all, or it might use a collection of interconnected class instances to represent a single abstract game object. Whatever its design, the runtime object model must provide a faithful reproduction of the object types, attributes and behaviors advertised by the world editor.

The runtime object model is the in-game manifestation of the abstract tool-side object model presented to the designers in the world editor. Designs vary widely, but most game engines follow one of two basic architectural styles:

- *Object-centric*. In this style, each tool-side game object is represented at runtime by a single class instance or a small collection of interconnected instances. Each object has a set of *attributes* and *behaviors* that are encapsulated within the class (or classes) of which the object is an instance. The game world is just a collection of game objects.
- *Property-centric*. In this style, each tool-side game object is represented only by a unique id (implemented as an integer, hashed string id or string). The *properties* of each game object are distributed across many data tables, one per property type, and keyed by object id (rather than being centralized within a single class instance or collection of interconnected instances). The properties themselves are often implemented as instances of hard-coded classes. The *behavior* of a game object is implicitly defined by the collection of properties of which it is composed. For example, if an object has the "Health" property, then it can be damaged, lose health and eventually die. If an object has the "MeshInstance" property, then it can be rendered in 3D as an instance of a triangle mesh.

There are distinct advantages and disadvantages to each of these architectural styles. We'll investigate each one in some detail and note where one style has significant potential benefits over the other as they arise.

16.2.1 Object-Centric Architectures

In an *object-centric* game world object architecture, each logical game object is implemented as an instance of a class, or possibly a collection of interconnected class instances. Under this broad umbrella, many different designs are possible. We'll investigate a few of the most common designs in the following sections.

16.2.1.1 A Simple Object-Based Model in C: Hydro Thunder

Game object models needn't be implemented in an object-oriented language like C++ at all. For example, the arcade hit *Hydro Thunder*, by Midway Home Entertainment in San Diego, was written entirely in C. *Hydro* employed a very simple game object model consisting of only a few object types:

- boats (player- and AI-controlled),
- floating blue and red boost icons,
- ambient animated objects (animals on the side of the track, etc.),
- the water surface,
- ramps,
- waterfalls,
- particle effects,
- race track sectors (two-dimensional polygonal regions connected to one another that together define the watery region in which boats could race),
- static geometry (terrain, foliage, buildings along the sides of the track, etc.), and
- two-dimensional heads-up display (HUD) elements.

A few screenshots of *Hydro Thunder* are shown in Figure 16.1. Notice the hovering boost icons in both screenshots and the shark swimming by in the left image (an example of an ambient animated object).

Hydro had a C struct named `World_t` that stored and managed the contents of a game world (i.e., a single race track). The world contained pointers to arrays of various kinds of game objects. The static geometry was a single mesh instance. The water surface, waterfalls and particle effects were each represented by custom data structures. The boats, boost icons and other dynamic objects in the game were represented by instances of a general-purpose struct called `WorldOb_t` (i.e., a world object). This was *Hydro*'s equivalent of a *game object* as we've defined it in this chapter.

Figure 16.1. Screenshots from the arcade game *Hydro Thunder*, developed by Midway Home Entertainment in San Diego.

The `WorldOb_t` data structure contained data members encoding the position and orientation of the object, the 3D mesh used to render it, a set of collision spheres, simple animation state information (*Hydro* only supported rigid hierarchical animation), physical properties like velocity, mass and buoyancy, and other data common to all of the dynamic objects in the game. In addition, each `WorldOb_t` contained three pointers: a `void*` "user data" pointer, a pointer to a custom "update" function and a pointer to a custom "draw" function. So while *Hydro Thunder* was not object-oriented in the strictest sense, the *Hydro* engine did extend its non-object-oriented language (C) to support rudimentary implementations of two important OOP features: *inheritance* and *polymorphism*. The user data pointer permitted each type of game object to maintain custom state information specific to its type while inheriting the features common to all world objects. For example, the *Banshee* boat had a different booster mechanism than the *Rad Hazard*, and each booster mechanism required different state information to manage its deployment and stowing animations. The two function pointers acted like *virtual functions*, allowing world objects to have polymorphic behaviors (via their "update" functions) and polymorphic visual appearances (via their "draw" functions).

```
struct WorldOb_s
{
    Orient_t m_transform;    /* position/rotation */
    Mesh3d*  m_pMesh;        /* 3D mesh */
    /* ... */
    void*    m_pUserData;    /* custom state */

    void     (*m_pUpdate)(); /* polymorphic update */
    void     (*m_pDraw)();   /* polymorphic draw */
};
typedef struct WorldOb_s WorldOb_t;
```

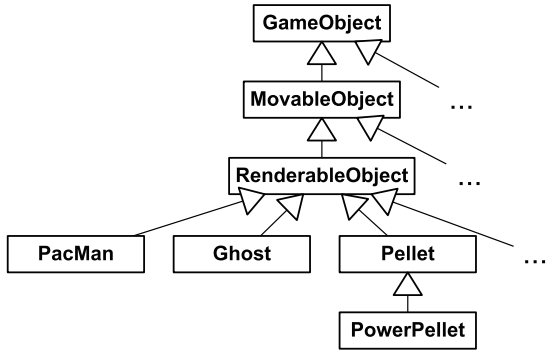

Figure 16.2. A hypothetical class hierarchy for the game *Pac-Man*.

16.2.1.2 Monolithic Class Hierarchies

It's natural to want to classify game object types taxonomically. This tends to lead game programmers toward an object-oriented language that supports inheritance. A class hierarchy is the most intuitive and straightforward way to represent a collection of interrelated game object types. So it is not surprising that the majority of commercial game engines employ a class hierarchy based technique.

Figure 16.2 shows a simple class hierarchy that could be used to implement the game *Pac-Man*. This hierarchy is rooted (as many are) at a common class called `GameObject`, which might provide some facilities needed by all object types, such as RTTI or serialization. The `MovableObject` class represents any object that has a position and orientation. `RenderableObject` gives the object an ability to be rendered (in the case of traditional *Pac-Man*, via a sprite, or in the case of a modern 3D *Pac-Man* game, perhaps via a triangle mesh). From `RenderableObject` are derived classes for the ghosts, Pac-Man, pellets and power pills that make up the game. This is just a hypothetical example, but it illustrates the basic ideas that underlie most game object class hierarchies—namely that common, generic functionality tends to exist at the root of the hierarchy, while classes toward the leaves of the hierarchy tend to add increasingly specific functionality.

A game object class hierarchy usually begins small and simple, and in that form, it can be a powerful and intuitive way to describe a collection of game object types. However, as class hierarchies grow, they have a tendency to deepen and widen simultaneously, leading to what I call a *monolithic class hierarchy*. This kind of hierarchy arises when virtually all classes in the game object model inherit from a single, common base class. The Unreal Engine's game object model is a classic example, as Figure 16.3 illustrates.

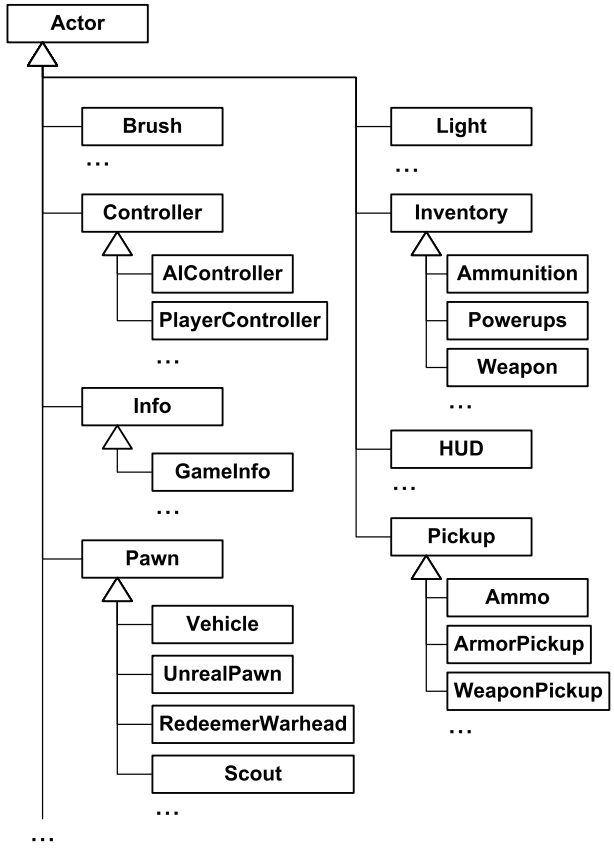

Figure 16.3. An excerpt from the game object class hierarchy in the *Unreal* engine.

16.2.1.3 Problems with Deep, Wide Hierarchies

Monolithic class hierarchies tend to cause problems for the game development team for a wide range of reasons. The deeper and wider a class hierarchy grows, the more extreme these problems can become. In the following sections, we'll explore some of the most common problems caused by wide, deep class hierarchies.

Understanding, Maintaining and Modifying Classes

The deeper a class lies within a class hierarchy, the harder it is to understand, maintain and modify. This is because to understand a class, you really need to understand all of its parent classes as well. For example, modifying the

behavior of an innocuous-looking virtual function in a derived class could violate the assumptions made by any one of the many base classes, leading to subtle, difficult-to-find bugs.

Inability to Describe Multidimensional Taxonomies

A hierarchy inherently classifies objects according to a particular system of criteria known as a *taxonomy*. For example, *biological taxonomy* (also known as *alpha taxonomy*) classifies all living things according to genetic similarities, using a tree with eight levels: domain, kingdom, phylum, class, order, family, genus and species. At each level of the tree, a different criterion is used to divide the myriad life forms on our planet into more and more refined groups.

One of the biggest problems with any hierarchy is that it can only classify objects along a single "axis"—according to one particular set of criteria—at each level of the tree. Once the criteria have been chosen for a particular hierarchy, it becomes difficult or impossible to classify along an entirely different set of "axes." For example, biological taxonomy classifies objects according to genetic traits, but it says nothing about the colors of the organisms. In order to classify organisms by color, we'd need an entirely different tree structure.

In object-oriented programming, this limitation of hierarchical classification often manifests itself in the form of wide, deep and confusing class hierarchies. When one analyzes a real game's class hierarchy, one often finds that its structure attempts to meld a number of different classification criteria into a single class tree. In other cases, concessions are made in the class hierarchy to accommodate a new type of object whose characteristics were not anticipated when the hierarchy was first designed. For example, imagine the seemingly logical class hierarchy describing different types of vehicles, depicted in Figure 16.4.

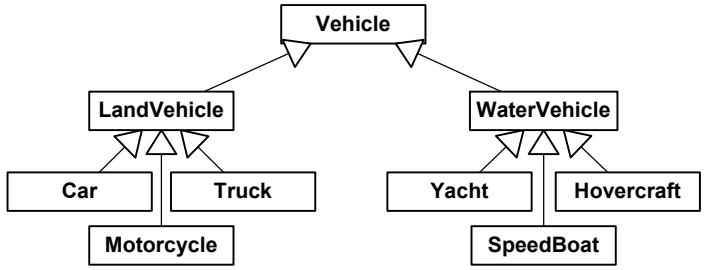

Figure 16.4. A seemingly logical class hierarchy describing various kinds of vehicles.

What happens when the game designers announce to the programmers that they now want the game to include an *amphibious* vehicle? Such a vehicle does not fit into the existing taxonomic system. This may cause the programmers to panic or, more likely, to "hack" their class hierarchy in various ugly and error-prone ways.

Multiple Inheritance: The Deadly Diamond

One solution to the amphibious vehicle problem is to utilize C++'s multiple inheritance (MI) features, as shown in Figure 16.5. At first glance, this seems like a good solution. However, multiple inheritance in C++ poses a number of practical problems. For example, multiple inheritance can lead to an object that contains multiple copies of its base class' members—a condition known as the "deadly diamond" or "diamond of death." (See Section 3.1.1.3 for more details.)

The difficulties in building an MI class hierarchy that works and that is understandable and maintainable usually outweigh the benefits. As a result, most game studios prohibit or severely limit the use of multiple inheritance in their class hierarchies.

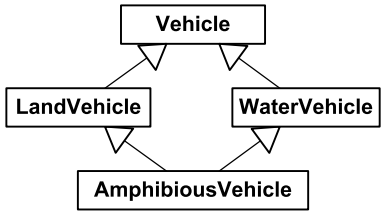

Figure 16.5. A diamond-shaped class hierarchy for amphibious vehicles.

Mix-In Classes

Some teams do permit a limited form of MI, in which a class may have any number of parent classes but only *one* grandparent. In other words, a class may inherit from one and only one class in the main inheritance hierarchy, but it may also inherit from any number of *mix-in classes* (stand-alone classes with no base class). This permits common functionality to be factored out into a mix-in class and then spot-patched into the main hierarchy wherever it is needed. This is shown in Figure 16.6. However, as we'll see below, it's usually better to *compose* or *aggregate* such classes than to *inherit* from them.

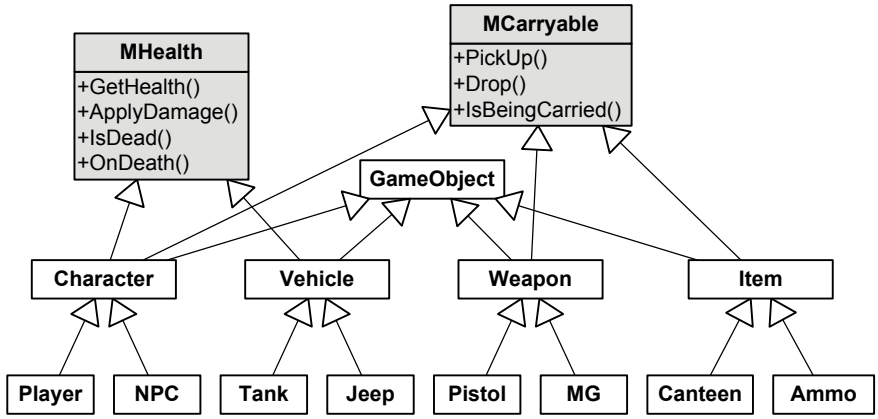

Figure 16.6. A class hierarchy with mix-in classes. The MHealth mix-in class adds the notion of health and the ability to be killed to any class that inherits it. The MCarryable mix-in class allows an object that inherits it to be carried by a Character.

The Bubble-Up Effect

When a monolithic class hierarchy is first designed, the root class or classes are usually very simple, each one exposing only a minimal feature set. However, as more and more functionality is added to the game, the desire to *share code* between two or more *unrelated* classes begins to cause features to "bubble up" the hierarchy.

For example, we might start out with a design in which only wooden crates can float in water. However, once our game designers see those cool floating crates, they begin to ask for other kinds of floating objects, like characters, bits of paper, vehicles and so on. Because "floating versus non-floating" was not one of the original classification criteria when the hierarchy was designed, the programmers quickly discover the need to add flotation to classes that are totally unrelated within the class hierarchy. Multiple inheritance is frowned upon, so the programmers decide to move the flotation code up the hierarchy, into a base class that is common to all objects that need to float. The fact that some of the classes that derive from this common base class *cannot float* is seen as less of a problem than duplicating the flotation code across multiple classes. (A Boolean member variable called something like `m_bCanFloat` might even be added to make the distinction clear.) The ultimate result is that flotation eventually becomes a feature of the root object in the class hierarchy (along with pretty much every other feature in the game).

The `Actor` class in Unreal is a classic example of this "bubble-up effect." It contains data members and code for managing rendering, animation, physics, world interaction, audio effects, network replication for multiplayer games,

object creation and destruction, actor iteration (i.e., the ability to iterate over all actors meeting a certain criteria and perform some operation on them), and message broadcasting. Encapsulating the functionality of various engine subsystems is difficult when features are permitted to "bubble up" to the root-most classes in a monolithic class hierarchy.

16.2.1.4 Using Composition to Simplify the Hierarchy

Perhaps the most prevalent cause of monolithic class hierarchies is over-use of the "is-a" relationship in object-oriented design. For example, in a game's GUI, a programmer might decide to derive the class `Window` from a class called `Rectangle`, using the logic that GUI windows are always rectangular. However, a window *is not a* rectangle—it *has a* rectangle, which defines its boundary. So a more workable solution to this particular design problem is to embed an instance of the `Rectangle` class inside the `Window` class, or to give the `Window` a pointer or reference to a `Rectangle`.

In object-oriented design, the "has-a" relationship is known as *composition*. In composition, a class A either contains an *instance* of class B directly, or contains a *pointer* or *reference* to an instance of B. Strictly speaking, in order for the term "composition" to be applicable, class A must *own* class B. This means that when an instance of class A is created, it automatically creates an instance of class B as well; when that instance of A is destroyed, its instance of B is destroyed, too. We can also link classes to one another via a pointer or reference *without* having one of the classes manage the other's lifetime. In that case, the technique is usually called *aggregation*.

Converting Is-A to Has-A

Converting "is-a" relationships into "has-a" relationships can be a useful technique for reducing the width, depth and complexity of a game's class hierarchy. To illustrate, let's take a look at the hypothetical monolithic hierarchy shown in Figure 16.7. The root `GameObject` class provides some basic functionality required by all game objects (e.g., RTTI, reflection, persistence via serialization, network replication, etc.). The `MovableObject` class represents any game object that has a *transform* (i.e., a position, orientation and optional scale). `RenderableObject` adds the ability to be rendered on-screen. (Not all game objects need to be rendered—for example, an invisible `TriggerRegion` class could be derived directly from `MovableObject`.) The `CollidableObject` class provides collision information to its instances. The `AnimatingObject` class grants to its instances the ability to be animated via a skeletal joint hierarchy. Finally, the `PhysicalObject` gives its instances the

ability to be physically simulated (e.g., a rigid body falling under the influence of gravity and bouncing around in the game world).

One big problem with this class hierarchy is that it limits our design choices when creating new types of game objects. If we want to define an object type that is physically simulated, we are forced to derive its class from `PhysicalObject` even though it may not require skeletal animation. If we want a game object class with collision, it must inherit from `CollidableObject` even though it may be invisible and hence not require the services of `RenderableObject`.

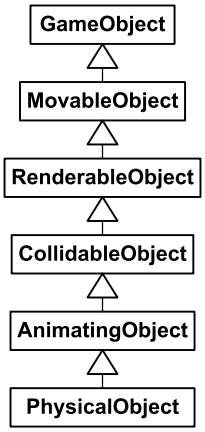

Figure 16.7. A hypothetical game object class hierarchy using only inheritance to associate the classes.

A second problem with the hierarchy shown in Figure 16.7 is that it is difficult to extend the functionality of the existing classes. For example, let's imagine we want to support morph target animation, so we derive two new classes from `AnimatingObject` called `SkeletalObject` and `MorphTargetObject`. If we wanted both of these new classes to have the ability to be physically simulated, we'd be forced to refactor `PhysicalObject` into two nearly identical classes, one derived from `SkeletalObject` and one from `MorphTargetObject`, or turn to multiple inheritance.

One solution to these problems is to isolate the various features of a `GameObject` into independent classes, each of which provides a single, well-defined service. Such classes are sometimes called *components* or *service objects*. A componentized design allows us to select only those features we need for each type of game object we create. In addition, it permits each feature to be maintained, extended or refactored without affecting the others. The individual components are also easier to understand, and easier to test, because they are decoupled from one another. Some component classes correspond directly to a single engine subsystem, such as rendering, animation, collision, physics, audio, etc. This allows these subsystems to remain distinct and well-encapsulated when they are integrated together for use by a particular game object.

Figure 16.8 shows how our class hierarchy might look after refactoring it into components. In this revised design, the `GameObject` class acts like a hub, containing pointers to each of the optional components we've defined. The `MeshInstance` component is our replacement for the `RenderableObject` class—it represents an instance of a triangle mesh and encapsulates the knowledge of how to render it. Likewise, the `AnimationController` component replaces `AnimatingObject`, exposing skeletal animation services to the `GameObject`. Class `Transform` replaces `MovableObject` by maintaining the position, orientation and scale of the object. The `RigidBody` class represents the collision geometry of a game object and provides its `GameObject` with an interface into the low-level collision and physics systems, replacing

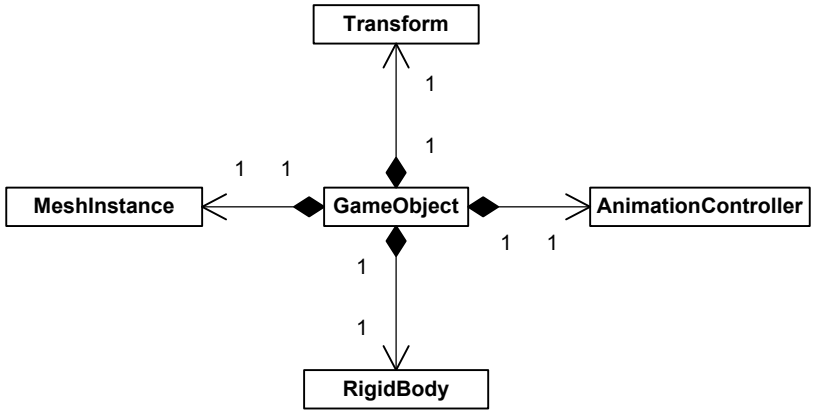

Figure 16.8. Our hypothetical game object class hierarchy, refactored to favor class composition over inheritance.

`CollidableObject` and `PhysicalObject`.

Component Creation and Ownership

In this kind of design, it is typical for the "hub" class to *own* its components, meaning that it manages their *lifetimes*. But how should a `GameObject` "know" which components to create? There are numerous ways to solve this problem, but one of the simplest is to provide the root `GameObject` class with pointers to all possible components. Each unique type of game object is defined as a derived class of `GameObject`. In the `GameObject` constructor, all of the component pointers are initially set to `nullptr`. Each derived class's constructor is then free to create whatever components it may need. For convenience, the default `GameObject` destructor can clean up all of the components automatically. In this design, the hierarchy of classes derived from `GameObject` serves as the primary taxonomy for the kinds of objects we want in our game, and the component classes serve as optional add-on features.

One possible implementation of the component creation and destruction logic for this kind of hierarchy is shown below. However, it's important to realize that this code is just an example—implementation details vary widely, even between engines that employ essentially the same kind of class hierarchy design.

```
class GameObject
{
protected:
```

```
    // My transform (position, rotation, scale).
    Transform            m_transform;

    // Standard components:
    MeshInstance*        m_pMeshInst;
    AnimationController* m_pAnimController;
    RigidBody*           m_pRigidBody;

public:
    GameObject()
    {
        // Assume no components by default.
        // Derived classes will override.
        m_pMeshInst = nullptr;
        m_pAnimController = nullptr;
        m_pRigidBody = nullptr;
    }

    ~GameObject()
    {
        // Automatically delete any components created by
        // derived classes. (Deleting null pointers OK.)
        delete m_pMeshInst;
        delete m_pAnimController;
        delete m_pRigidBody;
    }

    // ...
};

class Vehicle : public GameObject
{
protected:
    // Add some more components specific to Vehicles...
    Chassis* m_pChassis;
    Engine*  m_pEngine;
    // ...

public:
    Vehicle()
    {
        // Construct standard GameObject components.
        m_pMeshInst = new MeshInstance;
        m_pRigidBody = new RigidBody;

        // NOTE: We'll assume the animation controller
        // must be provided with a reference to the mesh
```

```
            // instance so that it can provide it with a
            // matrix palette.
            m_pAnimController
             = new AnimationController(*m_pMeshInst);

            // Construct vehicle-specific components.
            m_pChassis = new Chassis(*this,
                                     *m_pAnimController);

            m_pEngine = new Engine(*this);
        }

        ~Vehicle()
        {
            // Only need to destroy vehicle-specific
            // components, as GameObject cleans up the
            // standard components for us.
            delete m_pChassis;
            delete m_pEngine;
        }
    };
```

16.2.1.5 Generic Components

Another more flexible (but also trickier to implement) alternative is to provide the root game object class with a generic linked list of components. The components in such a design usually all derive from a common base class—this allows us to iterate over the linked list and perform polymorphic operations, such as asking each component what type it is or passing an event to each component in turn for possible handling. This design allows the root game object class to be largely oblivious to the component types that are available and thereby permits new types of components to be created without modifying the game object class in many cases. It also allows a particular game object to contain an arbitrary number of instances of each type of component. (The hard-coded design permits only a fixed number, determined by how many pointers to each component exist within the game object class.)

This kind of design is illustrated in Figure 16.9. It is trickier to implement than a hard-coded component model because the game object code must be written in a totally generic way. The component classes can likewise make no assumptions about what other components might or might not exist within the context of a particular game object. The choice between hard-coding the component pointers or using a generic linked list of components is not an easy one to make. Neither design is clearly superior—they each have their pros and cons, and different game teams take different approaches.

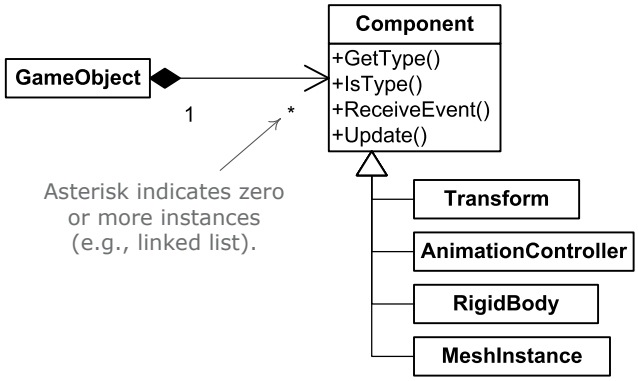

Figure 16.9. A linked list of components can provide flexibility by allowing the hub game object to be unaware of the details of any particular component.

16.2.1.6 Pure Component Models

What would happen if we were to take the componentization concept to its extreme? We would move literally *all* of the functionality out of our root `GameObject` class into various component classes. At this point, the game object class would quite literally be a behavior-less container, with a unique id and a bunch of pointers to its components, but otherwise containing no logic of its own. So why not eliminate the class entirely? One way to do this is to give each component a copy of the game object's unique id. The components are now linked together into a logical grouping by id. Given a way to quickly look up any component by id, we would no longer need the `GameObject` "hub" class at all. I will use the term *pure component model* to describe this kind of architecture. It is illustrated in Figure 16.10.

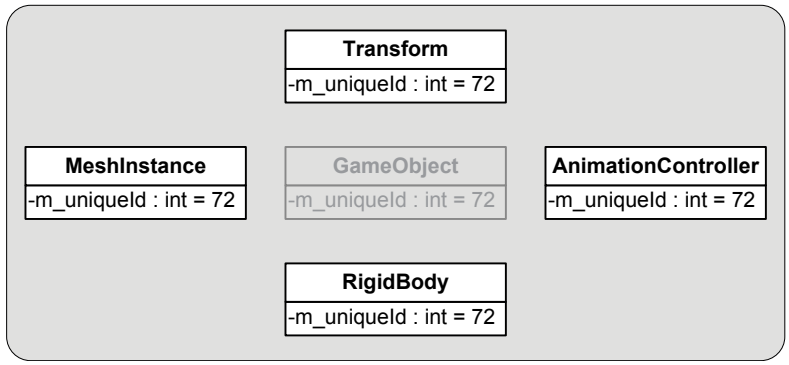

Figure 16.10. In a pure component model, a logical game object is comprised of many components, but the components are linked together only indirectly, by sharing a unique id.

A pure component model is not quite as simple as it first sounds, and it is not without its share of problems. For one thing, we still need some way of defining the various concrete types of game objects our game needs and then arranging for the correct component classes to be instantiated whenever an instance of the type is created. Our `GameObject` hierarchy used to handle construction of components for us. Instead, we might use a factory pattern, in which we define factory classes, one per game object type, with a virtual construction function that is overridden to create the proper components for each game object type. Or we might turn to a data-driven model, where the game object types are defined in a text file that can be parsed by the engine and consulted whenever a type is instantiated.

Another issue with a components-only design is inter-component communication. Our central `GameObject` acted as a "hub," marshalling communications between the various components. In pure component architectures, we need an efficient way for the components making up a single game object to talk to one another. This could be done by having each component look up the other components using the game object's unique id. However, we probably want a much more efficient mechanism—for example the components could be prewired into a circular linked list.

In the same sense, sending messages from one game object to another is difficult in a pure componentized model. We can no longer communicate with the `GameObject` instance, so we either need to know a priori with which component we wish to communicate, or we must multicast to all components that make up the game object in question. Neither option is ideal.

Pure component models can and have been made to work on real game projects. These kinds of models have their pros and cons, but again, they are not clearly better than any of the alternative designs. Unless you're part of a research and development effort, you should probably choose the architecture with which you are most comfortable and confident, and which best fits the needs of the particular game you are building.

16.2.2 Property-Centric Architectures

Programmers who work frequently in an object-oriented programming language tend to think naturally in terms of objects that contain attributes (data members) and behaviors (methods, member functions). This is the *object-centric view*:

- Object1
    - Position = (0, 3, 15)
    - Orientation = (0, 43, 0)

- Object2
  - Position = $(-12, 0, 8)$
  - Health = 15
- Object3
  - Orientation = $(0, -87, 10)$

However, it is possible to think primarily in terms of the attributes, rather than the objects. We define the set of all properties that a game object might have. Then for each property, we build a table containing the values of that property corresponding to each game object that has it. The property values are keyed by the objects' unique ids. This is what we will call the *property-centric view*:

- Position
  - Object1 = $(0, 3, 15)$
  - Object2 = $(-12, 0, 8)$
- Orientation
  - Object1 = $(0, 43, 0)$
  - Object3 = $(0, -87, 10)$
- Health
  - Object2 = 15

Property-centric object models have been used very successfully on many commercial games, including *Deus Ex 2* and the *Thief* series of games. See Section 16.2.2.5 for more details on exactly how these projects designed their object systems.

A property-centric design is more akin to a relational database than an object model. Each attribute acts like a table in a relational database, with the game objects' unique id as the *primary key*. Of course, in object-oriented design, an object is defined not only by its *attributes*, but also by its *behavior*. If all we have are tables of properties, then where do we implement the behavior? The answer to this question varies somewhat from engine to engine, but most often the behaviors are implemented in one or both of the following places:

- in the properties themselves, and/or
- via script code.

Let's explore each of these ideas further.

16.2.2.1 Implementing Behavior via Property Classes

Each type of property can be implemented as a *property class*. Properties can be as simple as a single Boolean or floating-point value or as complex as a renderable triangle mesh or an AI "brain." Each property class can provide behaviors via its hard-coded methods (member functions). The overall behavior of a particular game object is determined by the aggregation of the behaviors of all its properties.

For example, if a game object contains an instance of the `Health` property, it can be damaged and eventually destroyed or killed. The `Health` object can respond to any attacks made on the game object by decrementing the object's health level appropriately. A property object can also communicate with other property objects within the same game object to produce cooperative behaviors. For example, when the `Health` property detects and responds to an attack, it could possibly send a message to the `AnimatedSkeleton` property, thereby allowing the game object to play a suitable hit reaction animation. Similarly, when the `Health` property detects that the game object is about to die or be destroyed, it can talk to the `RigidBodyDynamics` property to activate a physics-driven explosion or a "rag doll" dead body simulation.

16.2.2.2 Implementing Behavior via Script

Another option is to store the property values as raw data in one or more database-like tables and use script code to implement a game object's behaviors. Every game object could have a special property called something like `ScriptId`, which, if present, specifies the block of script code (script function, or script object if the scripting language is itself object-oriented) that will manage the object's behavior. Script code could also be used to allow a game object to respond to events that occur within the game world. See Section 16.8 for more details on event systems and Section 16.9 for a discussion of game scripting languages.

In some property-centric engines, a core set of hard-coded property classes is provided by the engineers, but a facility is provided allowing game designers and programmers to implement new property types entirely in script. This approach was used successfully on the *Dungeon Siege* project, for example.

16.2.2.3 Properties versus Components

It's important to note that many of the authors cited in Section 16.2.2.5 use the term "component" to refer to what I call a "property object" here. In Section 16.2.1.4, I used the term "component" to refer to a subobject in an object-centric design, which isn't quite the same as a property object.

However, property objects are very closely related to components in many ways. In both designs, a single logical game object is made up of multiple subobjects. The main distinction lies in the roles of the subobjects. In a property-centric design, each subobject defines a particular attribute of the game object itself (e.g., health, visual representation, inventory, a particular magic power, etc.), whereas in a component-based (object-centric) design, the subobjects often represent linkages to particular low-level engine subsystems (renderer, animation, collision and dynamics, etc.) This distinction is so subtle as to be virtually irrelevant in many cases. You can call your design a *pure component model* (Section 16.2.1.6) or a *property-centric design* as you see fit, but at the end of the day, you'll have essentially the same result—a logical game object that is comprised of, and derives its behavior from, a collection of subobjects.

16.2.2.4 Pros and Cons of Property-Centric Designs

There are a number of potential benefits to an attribute-centric approach. It tends to be more memory-efficient, because we need only store attribute data that is actually in use (i.e., there are never game objects with unused data members). It is also easier to construct such a model in a data-driven manner—designers can define new attributes easily, without recompiling the game, because there are no game object class definitions to be changed. Programmers need only get involved when entirely new types of properties need to be added (presuming the property cannot be added via script).

A property-centric design can also be more cache-friendly than an object-centric model, because data of the same type is stored *contiguously* in memory. This is a commonplace optimization technique on modern gaming hardware, where the cost of accessing memory is far higher than the cost of executing instructions and performing calculations. (For example, on the PlayStation 3, the cost of a single cache miss is equivalent to the cost of executing literally thousands of CPU instructions.) By storing data contiguously in RAM, we can reduce or eliminate cache misses, because when we access one element of a data array, a large number of its neighboring elements are loaded into the same cache line. This approach to data design is sometimes called the *struct of arrays* technique, in contrast to the more-traditional *array of structs* approach. The differences between these two memory layouts are illustrated by the code snippet below. (Note that we wouldn't really implement a game object model in exactly this way—this example is meant only to illustrate the way in which a property-centric design tends to produce many contiguous arrays of like-typed data, rather than a single array of complex objects.)

```
static const U32 MAX_GAME_OBJECTS = 1024;
```

```
// Traditional array-of-structs approach.

struct GameObject
{
    U32         m_uniqueId;
    Vector      m_pos;
    Quaternion  m_rot;
    float       m_health;

    // ...
};

GameObject g_aAllGameObjects[MAX_GAME_OBJECTS];

// Cache-friendlier struct-of-arrays approach.

struct AllGameObjects
{
    U32         m_aUniqueId[MAX_GAME_OBJECTS];
    Vector      m_aPos[MAX_GAME_OBJECTS];
    Quaternion  m_aRot[MAX_GAME_OBJECTS];
    float       m_aHealth[MAX_GAME_OBJECTS];

    // ...
};

AllGameObjects g_allGameObjects;
```

Attribute-centric models have their share of problems as well. For example, when a game object is just a grab bag of properties, it becomes much more difficult to enforce relationships between those properties. It can be hard to implement a desired large-scale behavior merely by cobbling together the fine-grained behaviors of a group of property objects. It's also much trickier to debug such systems, as the programmer cannot slap a game object into the watch window in the debugger in order to inspect all of its properties at once.

16.2.2.5 Further Reading

A number of interesting PowerPoint presentations on the topic of property-centric architectures have been given by prominent engineers in the game industry at various game development conferences.

- Rob Fermier, "Creating a Data Driven Engine," Game Developer's Conference, 2002.
- Scott Bilas, "A Data-Driven Game Object System," Game Developer's Conference, 2002.